

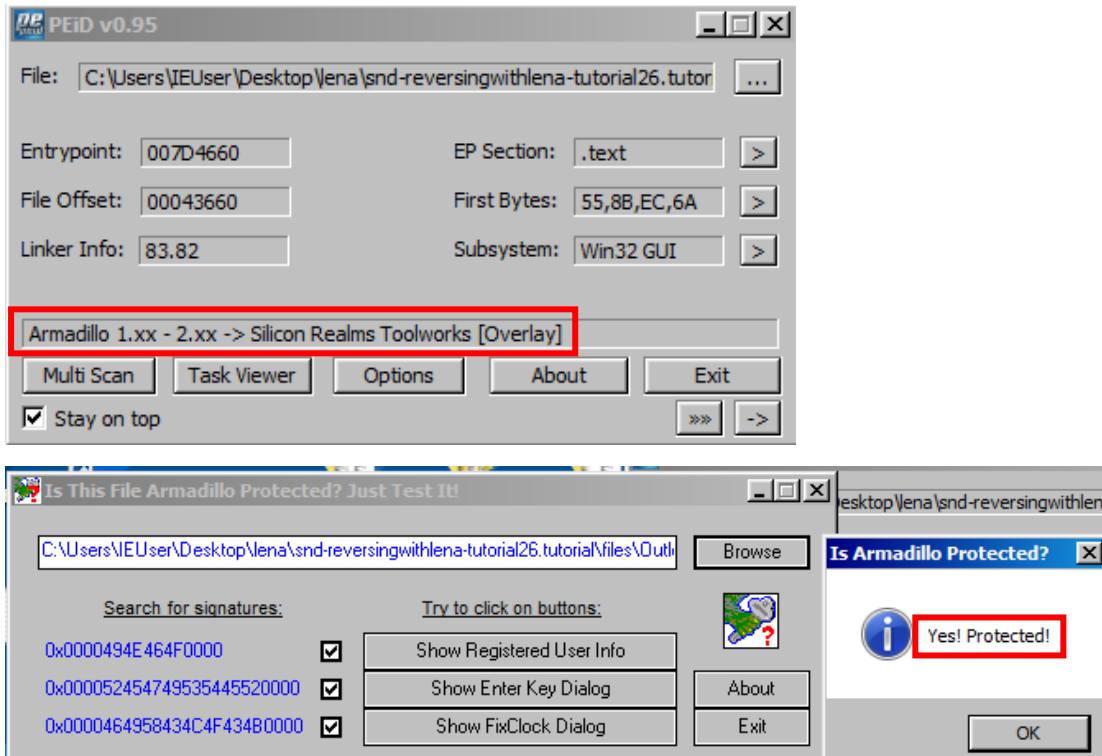
악성코드 분석 보고서

(sand-reversingwithlana-tutorials)

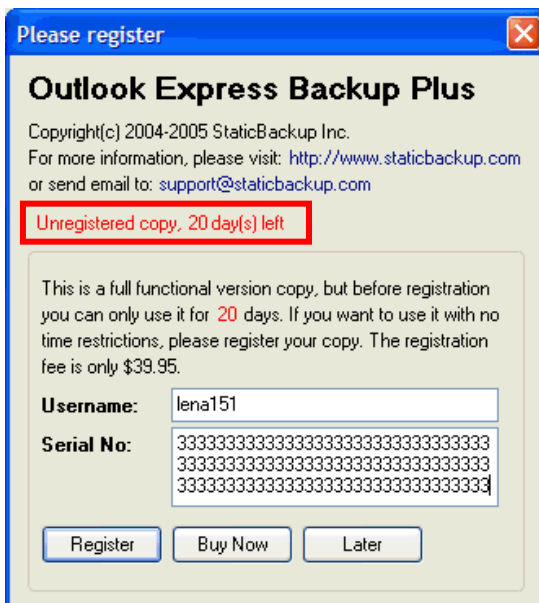
2025.11.10

24, 25와 마찬가지로 실행이 되지 않아 lena영상 및 다른 사용자의 글을 참고하여 작성하였다.

1) OutlookExpressBackup.exe – Loader 사용



PEiD와 IsItArma를 이용해서 확인해본 결과 아르마딜로 패커를 사용한 걸 볼 수 있다.
(버전은 정확하지 않을 수 있다.)



OutlookExpressBackup.exe를 실행했을 때 시리얼이 등록되지 않아서 남은 기간이 뜨는 걸 볼 수 있다.

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	Frame

0012CB10	00435CC2	RETURN to OutlookE.00435CC2		
0012CB14	0012CB5C	Pointer to next SEH record		
0012CB18	00435CDB	SE handler		
0012CB1C	0012CB34			
0012CB20	00460AFC	OutlookE.00460AFC		
0012CB24	01F5D244			
0012CB28	01F7EB30	ASCII "tPC"		
0012CB2C	00000000			
0012CB30	010004C6			
0012CB34	0012CB50			
0012CB38	00435C37	RETURN to OutlookE.00435C37 from OutlookE.00435C3C		
0012CB3C	00000000			
0012CB40	FFFFFFFF			
0012CB44	FFFFFFFF			
0012CB48	00000000			

해당 구간에서 Call Stack에 들어가보면 아무것도 없는 걸 볼 수 있는데 이럴 때 시스템 스택 창을 보면 0x00486D2C이 실행되기 전 코드를 볼 수 있다.

00435CBA	8B10	MOV EDX,DWORD PTR DS:[EAX]	
00435CBC	FF92 EC000000	CALL DWORD PTR DS:[EDX+EC]	OutlookE.00486D2C
00435CC2	8945 F8	MOV DWORD PTR SS:[EBP-8],EAX	
00435CC5	33C0	XOR EAX,EAX	
00435CC7	5A	POP EDX	OutlookE.00435CC2
00435CC8	59	POP ECX	OutlookE.00435CC2
00435CC9	59	POP ECX	OutlookE.00435CC2
00435CCA	64:8910	MOV DWORD PTR FS:[EAX],EDX	OutlookE.00435074

0x00435CC2로 넘어가보면 바로 위에 코드에서 EAX의 값을 받는 걸 볼 수 있다.

해당 함수를 호출하지 않으면 이전에 보았던 등록 실패 메시지 박스가 나타나지 않을 것이라고 추정했기 때문에 이전과 마찬가지로 해당 주소를 넘어가는 조건문을 찾아야 한다.

00435C30	55	PUSH EBP	
00435C3D	8BEC	MOV EBP,ESP	
00435C3F	83C4 F4	ADD ESP,-0C	
00435C42	53	PUSH EBX	
00435C43	56	PUSH ESI	
00435C44	66:894D FD	MOV WORD PTR SS:[EBP-3],CX	
00435C48	8855 FF	MOV BYTE PTR SS:[EBP-1],DL	
00435C4B	8B75 0C	MOV ESI,DWORD PTR SS:[EBP+C]	
00435C4E	8B5D 10	MOV EBX,DWORD PTR SS:[EBP+10]	
00435C51	66:8B4D FD	MOV CX,WORD PTR SS:[EBP-3]	
00435C55	8A55 FF	MOV DL,BYTE PTR SS:[EBP-1]	
00435C58	E8 1BFAFFFF	CALL OutlookE.00435678	
00435C5D	00435C5D	CALL OutlookE.00435C5D	

하지만 조건문을 찾을 수 없어서 0x00435C3C에 BP를 걸고 다시 실행해준다.

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	
0012CB38	00435C37	? OutlookE.00435C3C	OutlookE.00435C3C	
0012CB54	0052EF46	? OutlookE.00435C1C	OutlookE.0052EF41	
0012CB58	00000000	Arg1 = 00000000		

다시 실행 후 Call Stack을 보면 0x0052EF41에서 함수를 호출하는 걸 볼 수 있다.

0052EF10	6A 00	PUSH 0	
0052EF12	66:8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF18	B2 02	MOV DL,2	
0052EF1B	B8 A4EF5200	MOV EAX,OutlookE.0052EF94	ASCII "Thank you for your register, pl
0052EF20	E8 F76CF0FF	CALL OutlookE.0052EF94	
0052EF25	C783 4C020000 01000000	MOV DWORD PTR EAX,4C020000	
0052EF2F	EB 23	JMP SHORT OutlookE.0052EF31	
0052EF31	6A 00	PUSH 0	
0052EF33	66:8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF3A	B2 01	MOV DL,1	
0052EF3C	B8 DCE52000	MOV EAX,OutlookE.0052EF94	ASCII "Sorry, your registration inform
0052EF41	E8 D66CF0FF	CALL OutlookE.00435C1C	
0052EF44	8B83 1C030000	MOV EAX,DWORD PTR DS:[EBX+1C]	

It seems this is where we need to be

해당 함수로 넘어가보면 위에 성공했을 때 나오는 창을 확인할 수 있다. 그럼 이 부분에서 성공창과 실패창을 구분한다는 걸 알 수 있다.

0052EED0	59	POP ECX	OutlookE.00435C37
0052EED8	E8 A4F7FFFF	CALL OutlookE.0052E684	
0052EEE0	84C0	TEST AL,AL	
0052EEE2	74 4D	JE SHORT OutlookE.0052EF31	
0052EEE4	8D55 F0	LEA EDX,DWORD PTR SS:[EBP-10]	
0052EEE7	8B83 40030000	MOV EAX,DWORD PTR DS:[EBX+340]	
0052EEE9	E8 F2AAF3FF	CALL OutlookE.004699E4	
0052EEF2	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	
0052EEF5	50	PUSH EAX	OutlookE.0052EFDC
0052EEF6	8D55 EC	LEA EDX,DWORD PTR SS:[EBP-14]	
0052EEF9	8B83 1C030000	MOV EAX,DWORD PTR DS:[EBX+31C]	
0052EEFF	E8 E0AAF3FF	CALL OutlookE.004699E4	
0052EF04	8B55 EC	MOV EDX,DWORD PTR SS:[EBP-14]	
0052EF07	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
0052EF0A	59	POP ECX	OutlookE.00435C37
0052EF0B	E8 7CF8FFFF	CALL OutlookE.0052E78C	
0052EF10	6A 00	PUSH 0	
0052EF12	66 8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF19	B2 02	MOV DL,2	
0052EF1B	B8 A4EF5200	MOV EAX,OutlookE.0052EFA4	ASCII "Thank you for your register, pl
0052EF20	E8 F76CF0FF	CALL OutlookE.00435C1C	
0052EF25	C783 4C020000 01000000	MOV DWORD PTR DS:[EBX+24C],1	
0052EF2F	EB 23	JMP SHORT OutlookE.0052EF54	
0052EF31	6A 00	PUSH 0	
0052EF33	66 8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF3A	B2 01	MOV DL,1	
0052EF3C	B8 DCE52000	MOV EAX,OutlookE.0052EFD0	ASCII "Sorry, your registration inform
0052EF41	E8 D66CF0FF	CALL OutlookE.00435C1C	

JE분기문에서 성공창과 실패창을 구분하고 ZF = 0일 때 점프 안하는 걸 볼 수 있다. 즉 AL != 0 이어야 한다.

JE를 NOP처리하면 성공 창이 뜨겠지만 이러면 완전히 Register된게 아니라서 이런 식으로 바꾸면 안된다.

0x0052EEDB으로 들어가서 분석을 해줘야한다.

- ZF (Zero Flag): 결과가 0이면 1, 0이 아니면 0

- TEST AL, AL: AL and AL

AL = 0일 경우 0 and 0 == 0이므로 ZF = 1

AL = 1일 경우 1 and 1 == 1이므로 ZF = 0

- JE (Jump if Equal) == Jump if Zero

ZF = 1 이면 점프, ZF = 0 이면 점프 안 함

0052EED7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
0052EEDA	59	POP ECX	
0052EEDB	E8 A4F7FFFF	CALL OutlookE.0052E684	
0052EEEE	84C0	TEST AL,AL	
0052EEEE2	74 4D	JE SHORT OutlookE.0052EF31	
0052EEEE4	8D55 F0	LEA EDX,DWORD PTR SS:[EBP-10]	

0x0052EEDB에 BP를 걸어준 후 다시 실행하면 해당 구간에서 멈추고 'F7'로 해당 함수 들어가서 살펴본다.

0052E5F8	- FF25 84F9B800	JMP DWORD PTR DS:[B8F984]
0052E5FE	8BC0	MOV EAX, EAX
0052E600	- FF25 80F9B800	JMP DWORD PTR DS:[B8F980]
0052E606	8BC0	MOV EAX, EAX
0052E608	55	PUSH EBP
0052E609	8BEC	MOV EBP, ESP
0052E60B	6A 00	PUSH 0
0052E60D	53	PUSH EBX
0052E60E	33C0	XOR EAX, EAX
0052E610	55	PUSH EBP
0052E611	68 54E65200	PUSH OutlookE.0052E652
0052E616	64:FF30	PUSH DWORD PTR FS:[EAX]
0052E619	64:8920	MOV DWORD PTR FS:[EAX], ESP
0052E61C	8055 FC	LEA EDI, DWORD PTR SS:[EBP-4]
0052E61F	B8 6CE65200	MOV EAX, OutlookE.0052E66C
0052E624	E8 C70CEFF	CALL OutlookE.0040F2F0
0052E629	8B45 FC	MOV EAX, DWORD PTR SS:[EBP-4]
0052E62C	BA 7CE65200	MOV EDI, OutlookE.0052E67C
0052E631	E8 9A64EDFF	CALL OutlookE.00404AD0
0052E636	75 04	JNZ SHORT OutlookE.0052E63C
0052E638	B3 01	MOV BL, 1
0052E63A	EB 02	JMP SHORT OutlookE.0052E63E
0052E63C	33D8	XOR EBX, EBX
0052E63E	33C0	XOR EAX, EAX
0052E640	5A	POP EDI
0052E641	59	POP ECX
0052E642	59	POP ECX
0052E643	64:8910	MOV DWORD PTR FS:[EAX], EDI
0052E646	68 5BE65200	PUSH OutlookE.0052E65B
0052E64B	8045 FC	LEA EAX, DWORD PTR SS:[EBP-4]
0052E64E	E8 7160EDFF	CALL OutlookE.004046C4
0052E653	C3	RET
0052E654	^ F9 8759EDFF	JMP OutlookE.00403FF0

DS:[00B8F984]=01628947

We land on a far jump.
Press F8

해당 주소로 넘어오고 'F8'을 이용해서 넘어가면 JMP분기문이 있는 걸 볼 수 있다.
0x01628947로 넘어가는 걸 볼 수 있다.

01628947	6A 00	PUSH 0	
01628949	6A 00	PUSH 0	
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
01628953	E8 2BF0FFFF	CALL 01628783	
01628958	83C4 10	ADD ESP, 10	
0162895B	C2 0800	RET 8	
0162895E	6A 01	PUSH 1	
01628960	6A 00	PUSH 0	
01628962	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
01628966	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
0162896A	E8 14F0FFFF	CALL 01628783	
0162896F	83C4 10	ADD ESP, 10	

어디서 AL = 0이 되는지 찾기 위해 'F8'을 이용해서 차례로 실행해본다.

01628947	6A 00	PUSH 0	Registers (FPU) EAX 7FFDE000 ECX 0000083A EDX 7C90EB94 EBX 01F5D244
01628949	6A 00	PUSH 0	
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	
01628953	E8 2BF0FFFF	CALL 01628783	
01628958	83C4 10	ADD ESP, 10	
0162895B	C2 0800	RET 8	

0x01628958에서 AL = 0이 되는 걸 볼 수 있다. BP를 걸고 재실행 후 F7'을 이용해서 해당 함수를 들어가본다.

016288C7	EB 62	JMP SHORT 0162892B
016288C9	57	PUSH EDI
016288CA	B3 01	MOV BL,1
016288CC	E8 94FEFEFF	CALL 01618765
016288D1	59	POP ECX
016288D2	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
016288D5	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
016288DB	56	PUSH ESI
016288DC	E8 140BFFFF	CALL 016193F5
016288E1	84C0	TEST AL,AL
016288E3	75 04	JNZ SHORT 016288E9
016288E5	32DB	XOR BL,BL
016288E7	EB 15	JMP SHORT 016288FE
016288E9	807D 14 00	CMP BYTE PTR SS:[EBP+14],0
016288ED	74 0F	JE SHORT 016288FE
016288EF	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
016288F2	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
016288F8	56	PUSH ESI
016288F9	E8 B700FFFF	CALL 016189B5
016288FE	E8 6CFEFEFF	CALL 0161876F
01628903	57	PUSH EDI
01628904	8BF0	MOV ESI,EAX
01628906	E8 86FCFFFF	CALL 01628591
0162890B	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
01628911	E8 1F0BFFFF	CALL 01619435
01628916	56	PUSH ESI
01628917	E8 49FEFEFF	CALL 01618765
0162891C	59	POP ECX
0162891D	E8 4DFEFEFF	CALL 0161876F
01628922	50	PUSH EAX
01628923	FF15 34F16201	CALL DWORD PTR DS:[162F134]
01628929	8AC3	MOV AL,BL
0162892B	5F	POP EDI
0162892C	5E	POP ESI
0162892D	5B	POP EBX
0162892E	C9	LEAVE
0162892F	C3	RETN

'F8'을 이용해서 내려가보면 RETN이 보이고 어디서 AL이 바뀌는 지 알 수 있다. 중간에 조건문이랑 CALL함수들이 많지만 마지막에 MOV AL, BL로 인해 AL이 변하고 RETN이 되는 걸 볼 수 있다. 즉, 위에 부분은 건들지 않고 해당 부분만 바꾸면 해결되는 걸 볼 수 있다.

* 중간의 분기문들 해석

1. "016288DC" 함수 실행

1.1 리턴 값 '0' ==> 분기되지 않음

1.2 리턴 값 '1' ==> "016288E9" 주소로 분기

2. 분기되지 않은 경우 (1-1 수행 됨)

2.1 "BL" 레지스터 값을 '0'으로 세팅

2.2 "016288FE" 주소로 분기

2.3 "016288FE" 주소에서 "0162876F" 함수 호출

3. 분기된 경우 (1-2 수행 됨)

3.1 "[EBX+14] 값을 '0'과 비교

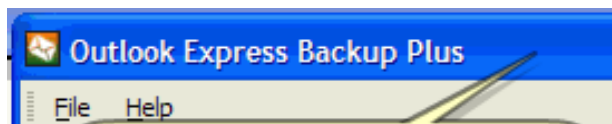
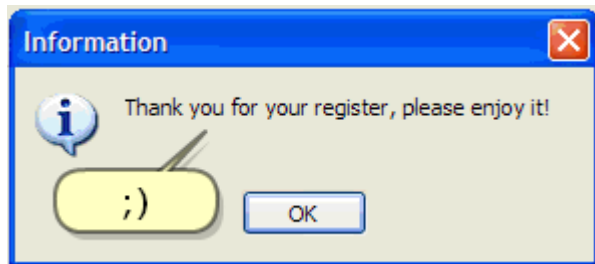
3.2 해당 값이 '0' ==> "0161876F" 주소로 분기

3.3 해당 값이 '0' 이 아님 ==> "016288 EF/F2/F8" 명령어 수행

3.3.1 "016288F9" 주소에서 "016289B5" 함수 호출

01628922	50	PUSH EHX
01628923	FF15 34F16201	CALL DWORD
01628929	B0 01	MOV AL, 1
0162892B	5F	POP EDI

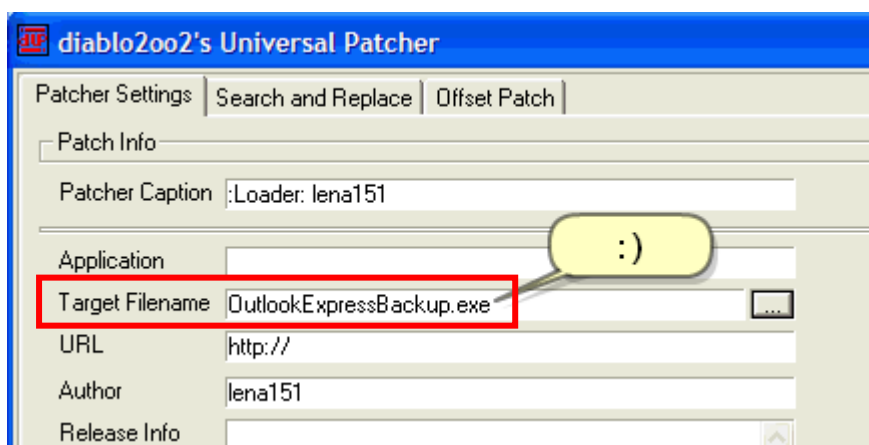
MOV AL, BL -> MOV AL, 1로 변경해주고 다시 실행해준다.



성공 창과 남은 시간이 없어진 걸 볼 수 있다.

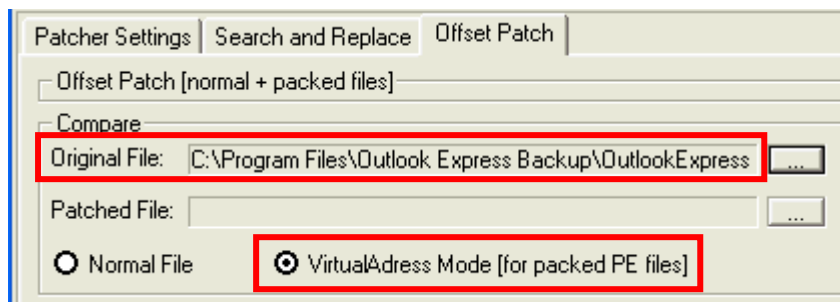
원래 OEP를 찾고 원본 코드에서 수정해줘야하는데 언패킹을 해주지 않고 수정을 했다. 그러면 패킹된 상태 이기 때문에 해당 파일을 다시 실행해보면 실행되지 않는다. (원래는 언패킹 => OEP 수정 => 임포트 테이블 수정)

이런 경우 로더를 이용해서 수정을 해줄거다. 하지만 여러 가지 이유 때문에 로더를 생성했던 시스템에서만 동작할 것이다. **가장 큰 이유는 프로그램이 실행될 때의 메모리 주소를 이용하여 패치하기 때문이다.** 실행되는 운영체제 환경마다 메모리 주소가 다르기 때문에 로더의 호환성을 높이려면 메인 프로그램의 코드 자체를 수정해야 한다.



"dUP" 이라는 로더 툴을 이용해서 수정해줄거다.

"Target Filename"에 수정해야 할 파일(OutlookExpressBackup.exe)을 선택해준다.



Offset Patch로 넘어가서 VirtualAddress Mode에 체크해주고 "Original File"에 원래 파일 (OutlookExpressBackup.exe)을 넣어준다.

```

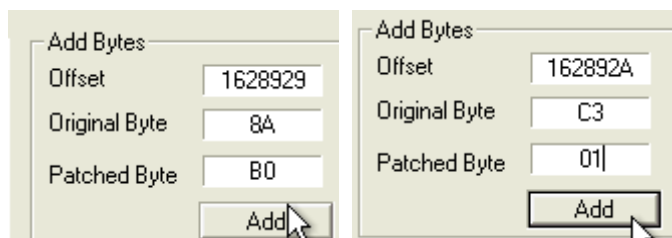
01628923  FF15 34F16201  CALL DWORD PTR DS:[162F134]
01628929  8AC3          MOV AL,BL
0162892B  5F           POP EDI

```

```

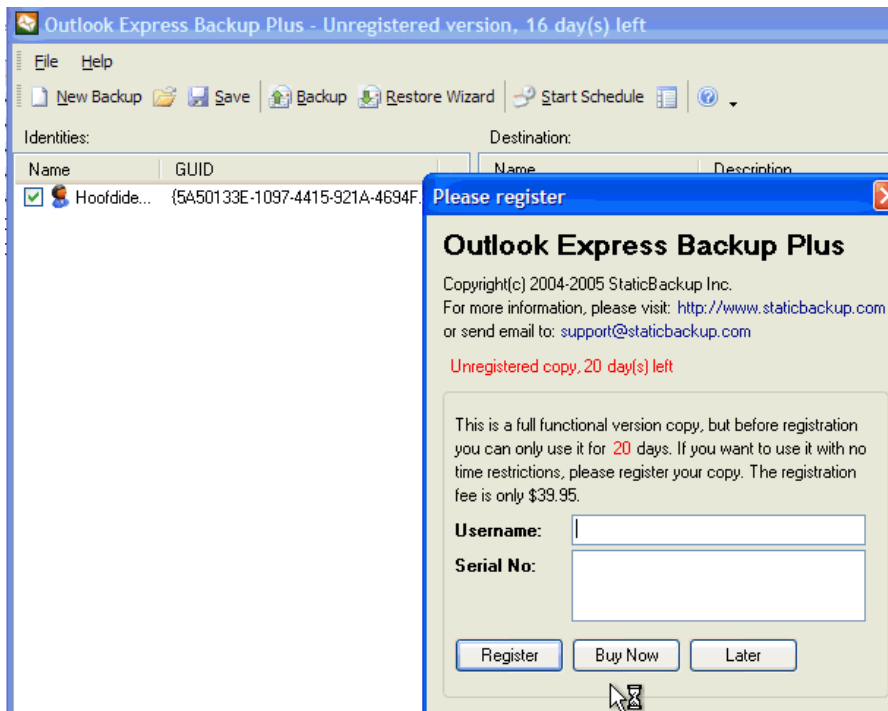
01628922  50           PUSH EAX
01628923  FF15 34F16201  CALL DWORD PTR DS:[162F134]
01628929  B0 01        MOV AL,1
0162892B  5F           POP EDI

```



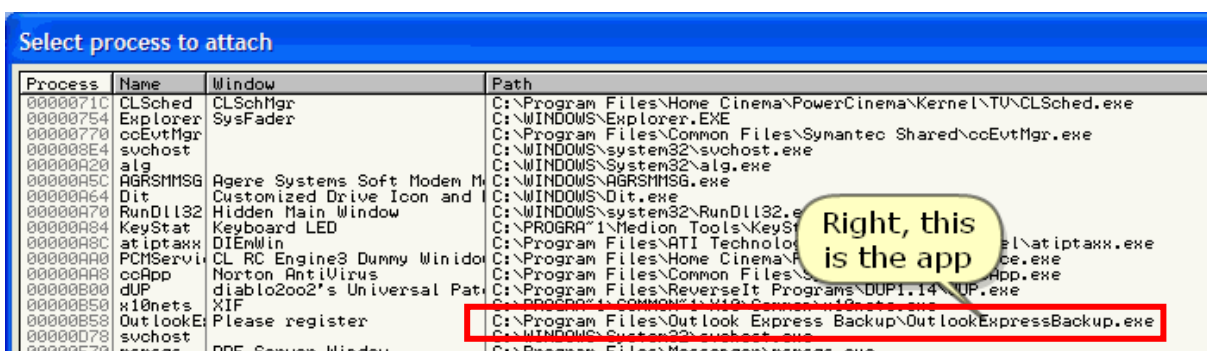
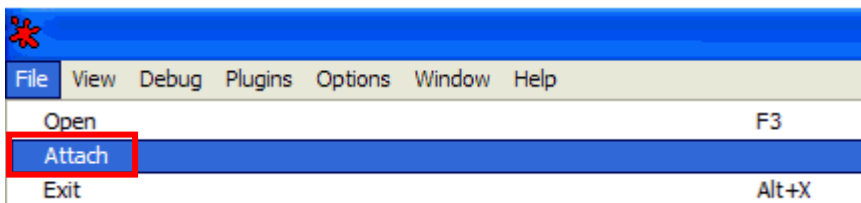
VA Offset	Original Byte	Patched Byte
1628929	8A	B0
162892A	C3	01

아까 수정한 부분을 1Byte씩 넣어서 수정해준다. 수정해 준 후 "Crate Loder"을 한 후 파일을 생성해준다.

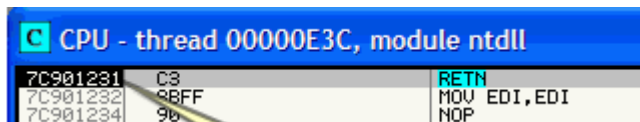


하지만 실행해보면 수정이 안 된 걸 볼 수 있다.

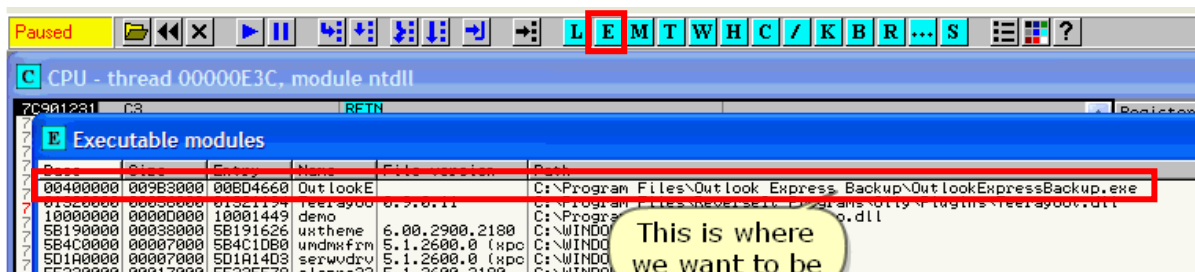
왜냐하면 "Armadillo" 프로텍터는 로더가 메모리 블록에 어떤 값을 작성하고자 하는 것을 탐지하여 "VA" 주소를 변경해버린다. 따라서 로더는 메인 프로그램을 패치할 수 없게 된다. 디버거에서 분석을 통해 어떤 부분이 문제고 어떻게 패치 하는지 설명하도록 하겠다.



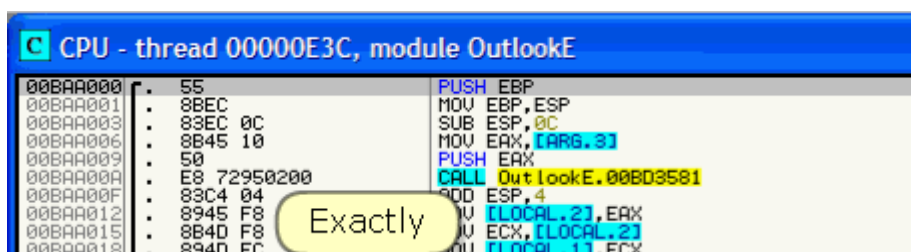
로더를 실행 시킨 후 올리디버그를 돌아가서 Attach를 해 준 후 OutlookExpressBackup.exe를 Attach해준다.



아직 진짜 프로그램 코드로 들어가기 전에 있다.



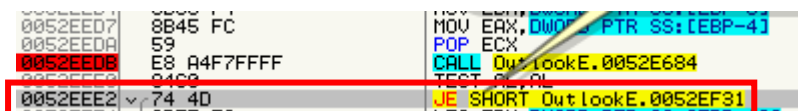
Executable modules로 들어가서 박스부분을 눌러준다.



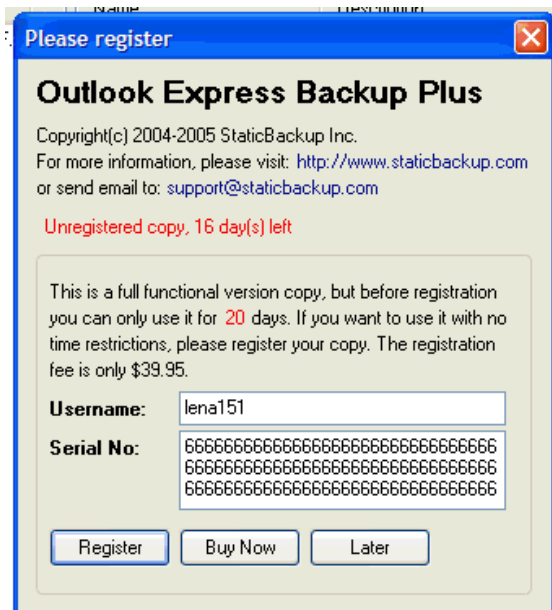
그럼 여기로 넘어가는 걸 볼 수 있다.



'F9'를 이용해서 실행해주면 레지스터 값이 아무것도 안보이는데 올리디버그가 혼란에 빠져서 그런거다 이 부분은 신경쓰지 않아도 된다.



처음 JE분기문이 있는 곳으로 넘어온다. (성공 창과 실패 창을 고르는 곳이 있는 곳)



다시 값들을 입력해주고 "Register"을 눌러준다.

0052EED8	59	POP ECX
0052EED9	E8 A4F7FFFF	CALL OutlookE.0052E684
0052EEE0	84C0	TEST AL,AL
0052EEE2	74 4D	JE SHORT OutlookE.0052EF31
0052EEE3	8BFF F8	MOV EBX,EBX

BP 구간에서 멈추는 걸 볼 수 있다.

0052E5F8	- FF25 84F9B800	JMP DWORD PTR DS:[B8F984]
0052E5FE	8BC0	MOV EAX,EAX

014A8947	6A 00	PUSH 0
014A8949	6A 00	PUSH 0
014A894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
014A894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
014A8953	E8 2BFFFFFF	CALL 014A8783
014A8958	83C4 10	ADD ESP,10

(현재)

01628947	6A 00	PUSH 0
01628949	6A 00	PUSH 0
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
01628953	E8 2BFFFFFF	CALL 01628783
01628958	83C4 10	ADD ESP,10

(전)

아까처럼 계속 넘어가보면 아까와 같은 0x0052E5F8에 도달하는 걸 볼 수 있다. 'F8'을 이용해서 넘어가본다. 근데 아까와 다르게 0x014A8947로 넘어가는 걸 볼 수 있다. (즉 주소 값이 달라진 걸 볼 수 있다.).

CALL이 부르는 함수 주소도 변경된 걸 볼 수 있다.

014A894B	FF7424 10	PUSH DWORD PTR
014A894F	FF7424 10	PUSH DWORD PTR
014A8953	E8 2BFFFFFF	CALL 014A8783
014A8958	83C4 10	ADD ESP,10
014A895C	C3 0000	RETN 0

해당 부분에 BP를 걸고 다시 실행해서 해당 함수에 들어가본다.

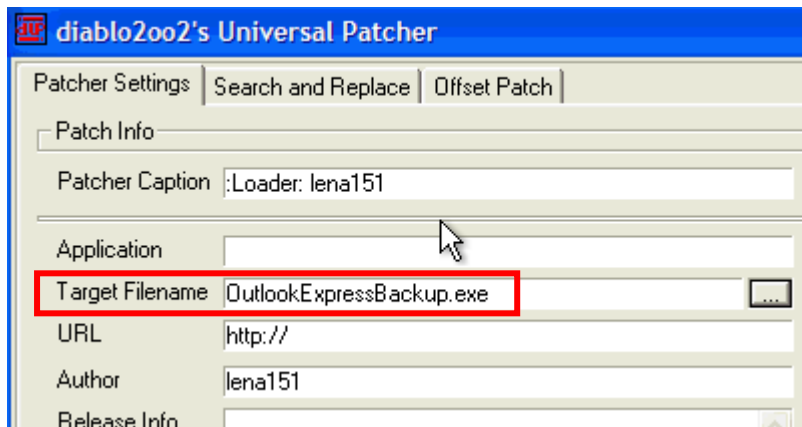
014A8917	50	49FEFEFF	PUSH ESI
014A891C	59		CALL 01498765
014A891D	59		POP ECX
014A891E	50	40FEFEFF	CALL 0149876F
014A8922	50		PUSH EAX
014A8923	50	FF15 34F14001	CALL DWORD PTR DS:[14AF134]
014A8929	8AC3		MOV AL,BL
014A892B	5F		POP EDI
014A892C	5E		POP ESI

(현재)

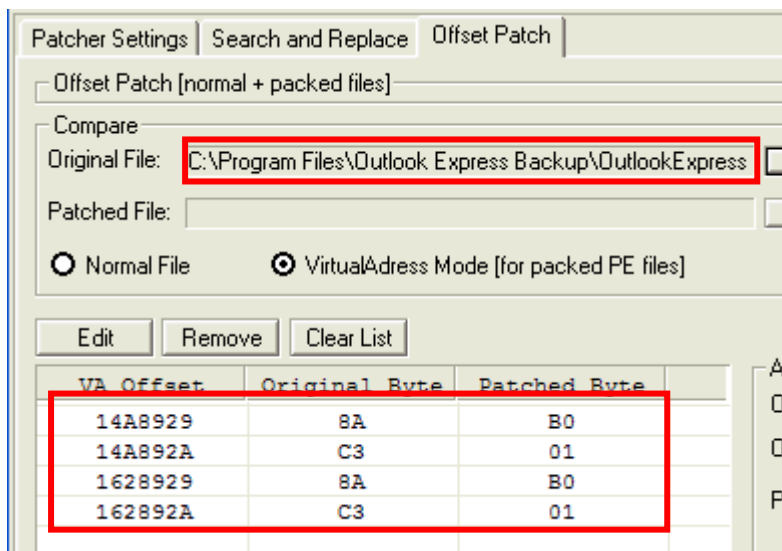
01628922	50		PUSH EAX
01628923	50	FF15 34F16201	CALL DWORD PTR DS:[162F134]
01628929	8AC3		MOV AL,BL
0162892B	5F		POP EDI
0162892C	5E		POP ESI

(전)

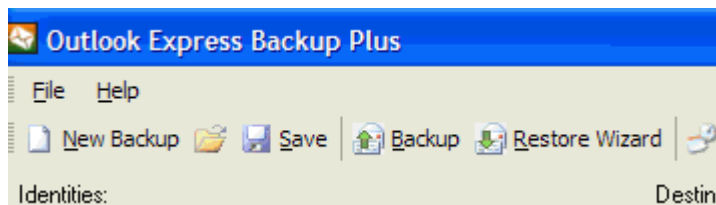
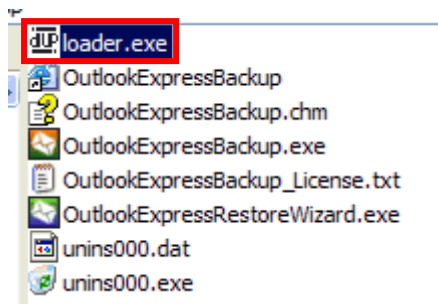
쪽 실행하다보면 아까와 주소가 다른 걸 볼 수 있다.



다시 “dUP” 로더를 이용해서 수정해줄거다. Target Filename에 수정할 파일을 넣어준다.(attach했기 때문에 해당 파일로 넣어주면 된다.)



로더 실행 시 변경된 VA 주소값과 이전과 동일한 주소 값을 패치해주면 된다.



로더를 실행해주면 모든 값이 변한 걸 볼 수 있다.

-> 레나가 첫 번째 로더의 이름을 loder.exe로 저장했는데 attach한 것은 OutlookExpressBackup.exe 이걸로 불러온 걸 볼 수 있다. 근데 왜 loder.exe가 아니라 OutlookExpressBackup.exe 이걸 실행했지,,? 그리고 두 번째 로더를 만들기 위해서 왜 Target Filename과 Original File을 둘 다 OutlookExpressBackup.exe 이걸로 했는지 잘 모르겠다,,, 보기 편하게 원래의 이름으로 바꾼건가,,

다른 사람이 작성한 설명을 보면

우선 로더를 통해 해당 바이너리를 실행한 상태에서 디버거에 "Attach" 시킨다. 그러면 디버거는 "Attach" 된 바이너리의 메모리 주소 값을 볼 수 있게 된다. 쪽쪽 분석하다 보면 우리가 "MOV AL,BL => MOV AL,1" 로 패치한 주소를 포함하는 코드 루틴으로 이동 될 것이다. 그런데 실행되는 코드는 같아도 주소가 다를 것이다. 즉 프로텍터가 로더 실행을 탐지하고 "VA" 주소를 변경하였기 때문에 절대 주소를 이용하는 기존의 로더는 무용지물이 된 것이다. 패치하는 방법은 의외로 간단한데 바로 로더에서 OPCODE 패치를 4 번하면 된다.

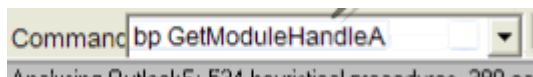
이런 식으로 설명되어 있는데 직접 실행해 볼 수 없어서 레나가 왜 저렇게 했는지 확인이 불가능하다,,,

2) OutlookExpressBackup.exe – Unpacking

레벨 25에서 "Armadillo" 프로텍터에 대하여 매뉴얼 언패킹을 다루었는데, 프로텍터의 적용 옵션에 따라 언패킹 방식이 약간씩 다르다. 이번 바이너리는 이전과 달리

"OutputDebugStringA" 함수가 사용되지 않았기 때문에 별다른 디버거 탐지 우회를 하지 않아도 된다. 본 프로텍터는 API 리다이렉션을 사용하기 때문에 이와 관련된 "Magic Jump" 조건문을 먼저 찾을 것이다.

* OutputDebugStringA가 없는지 어떻게 알았냐면 이전처럼 커맨드 플러그인을 이용하여 해당 함수에 BP를 설치하고 프로그램을 실행했으니 어느 주소에서도 멈추지 않았다.



Command에 bp GetModuleHandleA를 입력해서 BP를 걸어준다.

GetModuleHandleA : 현재 프로세스 안에 로드된 모듈(EXE 또는 DLL)의 핸들(HMODULE)을 반환하는 함수

여기서 핸들은 **그 모듈이 메모리에 로드된 시작 주소(=Base Address)**

HMODULE, HINSTANCE : 베이스 주소

파일 핸들, 프로세스 핸들, 윈도우 핸들 등은 베이스 주소가 아니다.

B Breakpoints			
Address	Module	Active	Disassembly
7C80B529	kernel32	Always	MOV EDI,EDI

이렇게 하나가 break된 걸 볼 수 있다. 해당 코드가 있는 곳으로 들어가본다.

7C80B529	8BFF	MOV EDI,EDI	ntdll.7C910738
7C80B52B	55	PUSH EBP	
7C80B52C	8BEC	MOV EBP,ESP	
7C80B52E	837D 08 00	CMP DWORD PTR SS:[EBP+8],0	
7C80B532	74 18	JE SHORT kernel32.7C80B54C	
7C80B534	FF75 08	PUSH DWORD PTR SS:[EBP+8]	OutlookE.<ModuleEntryPoint>
7C80B537	E8 682D0000	CALL kernel32.7C80E2A4	
7C80B53C	85C0	TEST EAX,EAX	
7C80B53E	74 08	JE SHORT kernel32.7C80B548	
7C80B540	FF70 04	PUSH DWORD PTR DS:[EAX+4]	
7C80B543	E8 F4300000	CALL kernel32.GetModuleHandleA	kernel32.7C816D4F
7C80B548	5D	POP EBP	
7C80B549	C2 0400	RETN 4	
7C80B54C	64:01 18000000	MOV EAX,DWORD PTR FS:[18]	
7C80B54E	8B48 2A	MOV EAX,DWORD PTR DS:[EAX+2A]	

GetModuleHandleA의 시작 부분으로 들어가진다.

BP가 CALL GetModuleHandleA 부분이 아니라 시작 부분으로 들어가진다. 왜냐하면 올리 디버거가 0x7C80B529에 BP(int3) 0xCC로 바꿔준다.

* MOV EDI, EDI : Microsoft가 API 핫패치(hotpatch) 기능 때문에 넣어놓는 고정 시그니처

7C80B529	8BFF	MOV EDI,EDI	ntdll.7C910738
7C80B52B	55	PUSH EBP	
7C80B52C	8BEC	MOV EBP,ESP	
7C80B52E	837D 08 00	CMP DWORD PTR SS:[EBP+8],0	
7C80B532	74 18	JE SHORT kernel32.7C80B54C	
7C80B534	FF75 08	PUSH DWORD PTR SS:[EBP+8]	Out Look E.<ModuleEntryPoint>
7C80B537	E8 682D0000	CALL kernel32.7C80E2A4	
7C80B53C	85C0	TEST EAX,EAX	
7C80B53E	74 08	JE SHORT kernel32.7C80B548	
7C80B540	FF70 04	PUSH DWORD PTR DS:[EAX+4]	
7C80B543	E8 F4300000	CALL kernel32.GetModuleHandleW	
7C80B548	5D	POP EBP	kernel32.7C816D4F
7C80B549	C2 0400	RETN 4	

아르마딜로는 API 엔트리 첫 바이트에 0xCC 같은 소프트웨어 BP가 있는지 검사하는 디버거 탐지 기능이 있기 때문에 BP를 RETN이 있는 마지막 부분으로 옮겨준다.

"Shift + F9"를 여러번 누르면서 "Magic Jump" 조건문을 찾아야 한다. 본인의 시스템에서는 총 13번을 눌러야지 "Maigc Jump" 조건문을 찾을 수 있었다.



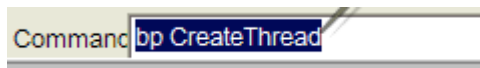
이렇게 RETN 4에 도달했을 때 해당 프로그램이 로딩이 되는데 지금 보이는 화면은 "진짜 프로그램"이 아니라 Armadillo 패커가 만든 임시 인트로 화면이다.

01539905	8B0D 40A15601	MOV ECX,DWORD PTR DS:[156A1401]	
0153990B	89040E	MOV DWORD PTR DS:[ESI+ECX],EAX	kernel32.7C800000
0153990E	A1 40A15601	MOV EAX,DWORD PTR DS:[156A1401]	
01539913	393C06	CMP DWORD PTR DS:[ESI+EAX],EDI	
01539916	75 16	JNZ SHORT 0153992E	
01539918	8D85 B4FEFFFF	LEA EAX,DWORD PTR SS:[EBP-14C]	
0153991E	50	PUSH EAX	kernel32.7C800000
0153991F	FF15 D4F05501	CALL DWORD PTR DS:[155F0D4]	kernel32.LoadLibraryA
01539925	8B0D 40A15601	MOV ECX,DWORD PTR DS:[156A1401]	
0153992B	89040E	MOV DWORD PTR DS:[ESI+ECX],EAX	kernel32.7C800000
0153992E	A1 40A15601	MOV EAX,DWORD PTR DS:[156A1401]	
01539933	393C06	CMP DWORD PTR DS:[ESI+EAX],EDI	
01539936	0F84 AC000000	JE 015399E8	
0153993C	33C9	XOR ECX,ECX	kernel32.7C80E82B

"RETN 4" 명령이 실행되면 "GetModuleHandleA" 함수를 호출했던 주소의 다음 주소로 이동된다. 기억할지 모르겠지만 위 사진과 같은 코드 루틴에서 제일 아래쪽의 "JE" 조건문이 API 리다이렉션을 해야 하는지 말아야할지 결정하는 조건문이다. 리다이렉션이 일어나지 않게 패치해야 한다.

0153992E	A1 40A15601	MOV EAX,DWORD PTR DS:[156A1401]	
01539933	393C06	CMP DWORD PTR DS:[ESI+EAX],EDI	
01539936	E9 AD000000	JMP 015399E8	
0153993B	90	NOP	
0153993C	33C9	XOR ECX,ECX	kernel32.7C80E82B
0153993E	8B03	MOV EAX,DWORD PTR DS:[EBX]	

JE 015399E8 -> JMP 015399E8로 바꿔준다. 그러면 리다이렉션을 하지 않는다.



원래 bp를 삭제하고 Command에 bp CreateThread를 입력해서 BP를 걸어준다.

CreateThread : 새로운 스레드(작업 흐름)를 만드는 함수

대부분의 패커는 언패킹(복호화) 루틴이 끝나면 새 스레드를 만들어 그 스레드에서 OEP를 실행시킨다.

B Breakpoints			
Address	Module	Active	Disassembly
7C81082F	kernel32	Always	MOV EDI,EDI

이렇게 하나가 break된 걸 볼 수 있다. 해당 코드가 있는 곳으로 들어가본다.

7C81082F	8BFF	MOV EDI,EDI	
7C810831	55	PUSH EBP	
7C810832	8BEC	MOV EBP,ESP	
7C810834	FF75 1C	PUSH DWORD PTR SS:[EBP+1C]	
7C810837	FF75 18	PUSH DWORD PTR SS:[EBP+18]	
7C81083A	FF75 14	PUSH DWORD PTR SS:[EBP+14]	
7C81083D	FF75 10	PUSH DWORD PTR SS:[EBP+10]	
7C810840	FF75 0C	PUSH DWORD PTR SS:[EBP+0C]	
7C810843	FF75 08	PUSH DWORD PTR SS:[EBP+08]	
7C810846	6A FF	PUSH -1	
7C810848	E8 D9FDFFFF	CALL kernel32.CreateRemoteThread	
7C81084D	5D	POP EBP	
7C81084E	C2 1800	RETN 18	
7C810851	90	NOP	
7C810852	90	NOP	

처음 0x7C81082F에 BP가 걸리는데 해당 부분 BP를 제거해주고 마지막에 BP를 걸어준다. 'F9'를 이용해서 해당 지점으로 온다.

0153F8B2	5E	POP ESI	Da
0153F8B3	C9	LEAVE	
0153F8B4	C3	RETN	
0153F8B5	55	PUSH EBP	

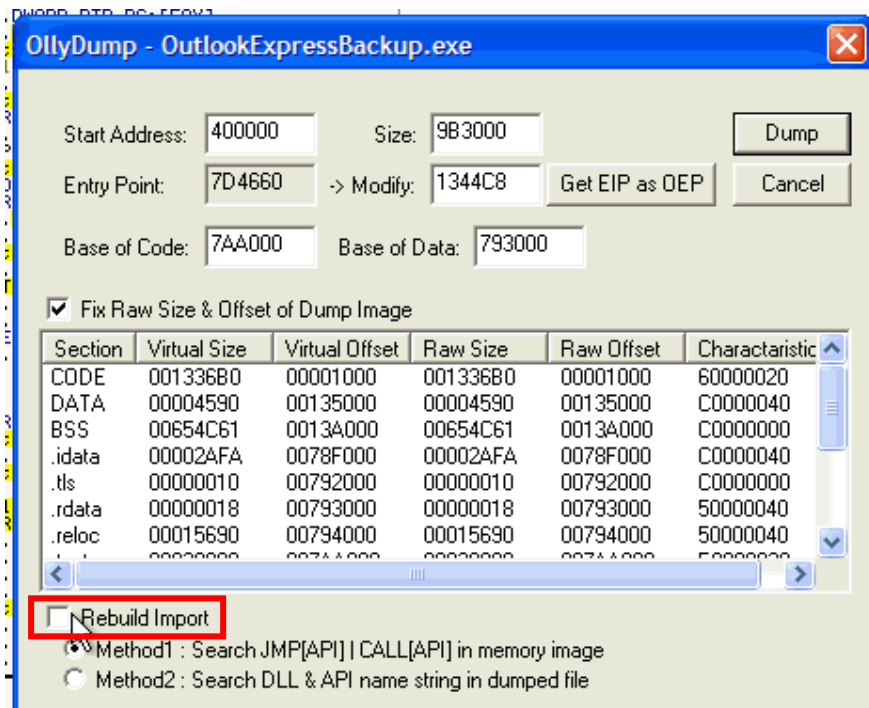
'F8'을 눌러서 해당 부분으로 넘어온 후 다시 RETN으로 가서 함수에서 나와준다.

01559343	A1 D8E65601	MOV EAX,DWORD PTR DS:[156E6D8]	
01559348	59	POP ECX	kernel32.7C8107FD
01559349	8B88 80000000	MOV ECX,DWORD PTR DS:[EAX+80]	
0155934F	3348 3C	XOR ECX,DWORD PTR DS:[EAX+3C]	
01559352	3348 30	XOR ECX,DWORD PTR DS:[EAX+30]	
01559355	F6C1 40	TEST CL,40	
01559358	75 08	JNZ SHORT 01559362	
0155935A	6A 01	PUSH 1	
0155935C	E8 D5D7FDFF	CALL 01536B36	
01559361	59	POP ECX	kernel32.7C8107FD
01559362	6A 00	PUSH 0	
01559364	C705 B8525601 D45F5601	MOV DWORD PTR DS:[15652B8],1565FD4	ASCII "RC"
0155936F	F8 F402FFFF	CALL 015399E8	
015593D0	E8 A3F3FEFF	CALL 01548775	
015593D2	50	PUSH EAX	
015593D3	A1 D8E65601	MOV EAX,DWORD PTR DS:[156E6D8]	
015593D8	8B48 74	MOV ECX,DWORD PTR DS:[EAX+74]	
015593DB	3348 3C	XOR ECX,DWORD PTR DS:[EAX+3C]	
015593DE	3348 10	XOR ECX,DWORD PTR DS:[EAX+10]	
015593E1	2BF9	SUB EDI,ECX	kernel32.7C8107FD
015593E3	FFD7	CALL EDI	
015593E5	8BD8	MOV EBX,EAX	
015593E7	EC	END	

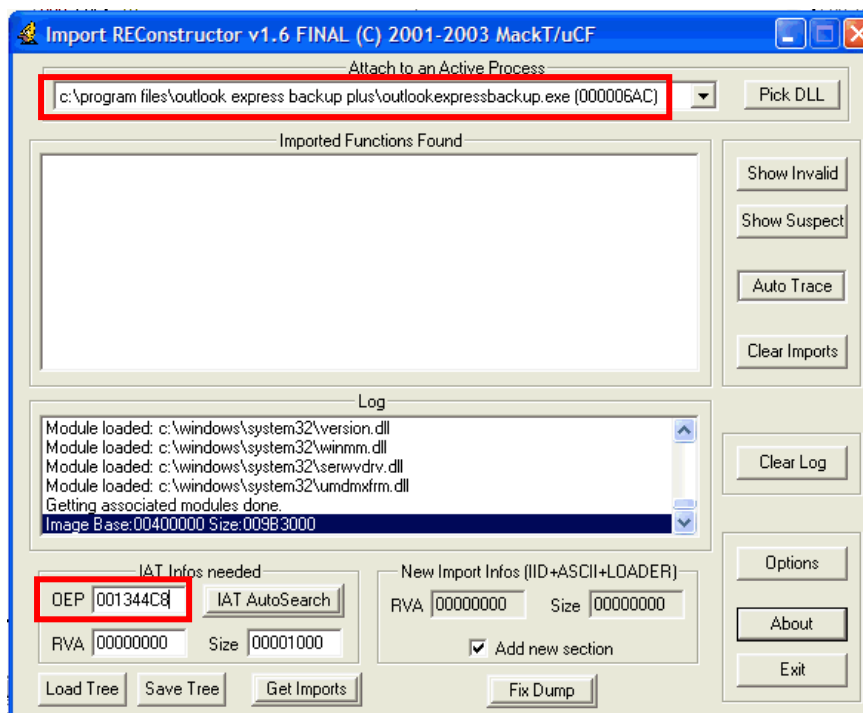
나오면 그 다음 코드로 넘어가는데 스크롤을 내려 보면 CALL EDI가 있는 걸 볼 수 있다. 이 부분이 OEP인 걸 볼 수 있다. -> OEP인 걸 아는 이유는 lena 25에 설명했다.

005344C8	55	PUSH EBP
005344C9	8BEC	MOV EBP,ESP
005344CA	83C4 F0	ADD ESP,-10
005344CB	B8 A83F5300	MOV EAX,Out lookE.00533FA8
005344CC	E8 0025EDFF	CALL Out lookE.00406AA8
005344CD	A1 30905300	MOV EAX,DWORD PTR DS:[539030]
005344CE	8B00	MOV EAX,DWORD PTR DS:[EAX]
005344CF	E8 985DF5FF	CALL Out lookE.0048A27C
005344D0	A1 30905300	MOV EAX,DWORD PTR DS:[539030]
005344D1	8B00	MOV EAX,DWORD PTR DS:[EAX]
005344D2	B8 8C465300	MOV EDI,Out lookE.0053468C
005344D3	E8 6F59F5FF	CALL Out lookE.00489E64
005344D4	B2 01	MOV DL,1
005344D5	A1 649F4A00	MOV EAX,DWORD PTR DS:[4A9F64]
005344D6	E8 AF96F7FF	CALL Out lookE.004AD8B0
005344D7	A3 5CECB800	MOV DWORD PTR DS:[B8EC5C],EAX
005344D8	33C0	XOR EAX,EAX

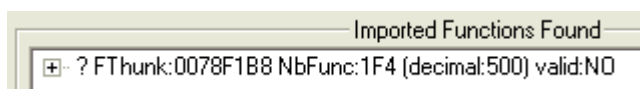
'F7'을 이용해서 들어가보면 OEP 시작 시그니처를 볼 수 있다. 해당 부분에서 덤프를 떠 준다.



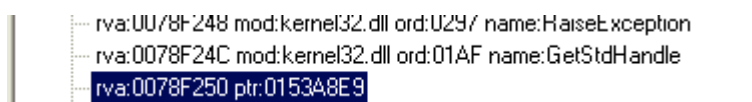
Rebuild Import에 체크를 해제 해주고 덤프를 뜬다.



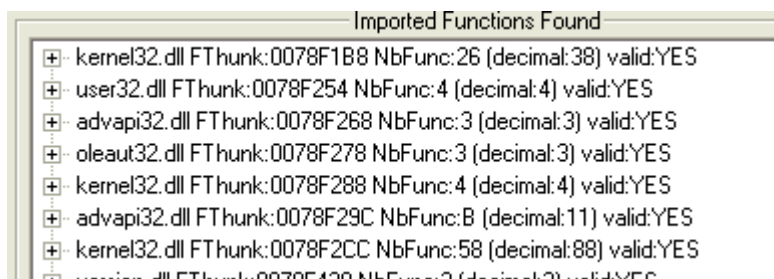
IAT를 새로 설정해줘야하기 때문에 Import REC를 이용해준다. OEP를 ImageBase를 뺀 값을 넣어주고 IAT AutoSearch를 해주고 Get Imports를 눌러준다.



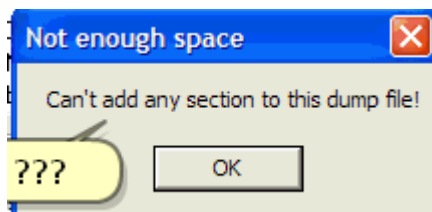
이렇게 Valid: NO가 나오는 걸 볼 수 있다.



들어가보면 문제가 되는 부분이 있는데 기억할지 모르겠지만 이전에 API 리다이렉션이 일어나지 않게 패치 하였기 때문에 위 사진에서 보여지는 "문제 되는 함수"들은 그냥 프로텍터가 추가한 것들이다. 그래서 그냥 없애주면 된다.



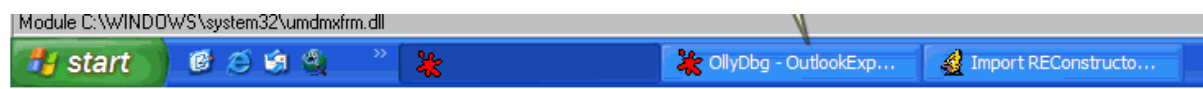
제거하면 모든 모듈이 YES로 바뀔 걸 볼 수 있다. Fix Dump를 눌러서 덤프를 떠준다.



그런데 해당 경고창이 나오는 걸 볼 수 있다. 이는 "Armadillo" 프로텍터에서 바이너리의 헤더를 변조해서 발생한 문제이다. 디버거의 메모리 맵 창에서 어떤 부분이 문제인지 봐야 한다.

Memory map							
Address	Size	Owner	Section	Contains	Type	Access	Initial access
003D0000	0000D000				Priv 00021004	RW	RW
003E0000	00001000				Map 00041004	RW	RW
003F0000	00003000				Priv 00021004	RW	RW
00400000	00001000	OutlookE			Imag 01001002	R	RWE
00401000	00134000	OutlookE	CODE	code	Imag 01001002	R	RWE
00535000	00005000	OutlookE	DATA	code	Imag 01001002	R	RWE
0053A000	00655000	OutlookE	BSS	code	Imag 01001002	R	RWE
00B8F000	00003000	OutlookE	.idata	code	Imag 01001002	R	RWE
00B92000	00001000	OutlookE	.tls	code	Imag 01001002	R	RWE
00B93000	00001000	OutlookE	.rdata	code	Imag 01001002	R	RWE
00B94000	00016000	OutlookE	.reloc	code	Imag 01001002	R	RWE
00B9A000	00030000	OutlookE	.text	code	Imag 01001002	R	RWE
00BDA000	00010000	OutlookE	.adata	code	Imag 01001002	R	RWE
00BEA000	00010000	OutlookE	.data	code,data,imports	Imag 01001002	R	RWE
00BFA000	00010000	OutlookE	.reloc1	code,relocations	Imag 01001002	R	RWE
00C0A000	000D0000	OutlookE	.pdata	code	Imag 01001002	R	RWE
00CDA000	000D9000	OutlookE	.rsrc	code,resources	Imag 01001002	R	RWE
00DC0000	00006000				Map 00041020	R E	R E
00E80000	00002000				Map 00041020	R E	R E

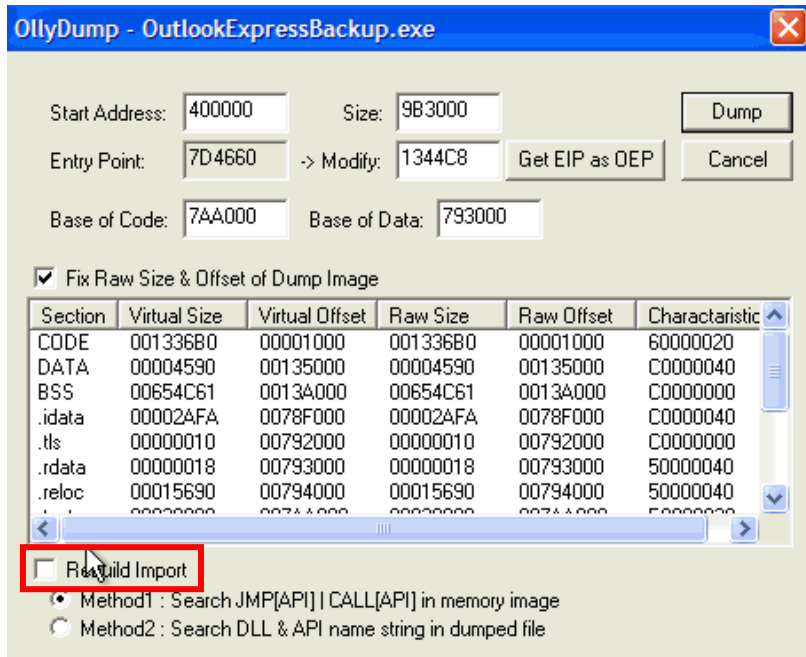
들어가보면 PE Header가 없는 걸 볼 수 있다. 이거는 손상이 되어 있기 때문에 올리디버가 인식을 못하기 때문에 이전 레벨에서 설명한 것처럼 원래 바이너리(패킹된 바이너리)의 헤더를 "Binary Copy" 하고, 현재 바이너리(언패킹 바이너리)에 "Binary Paste" 하면 된다.



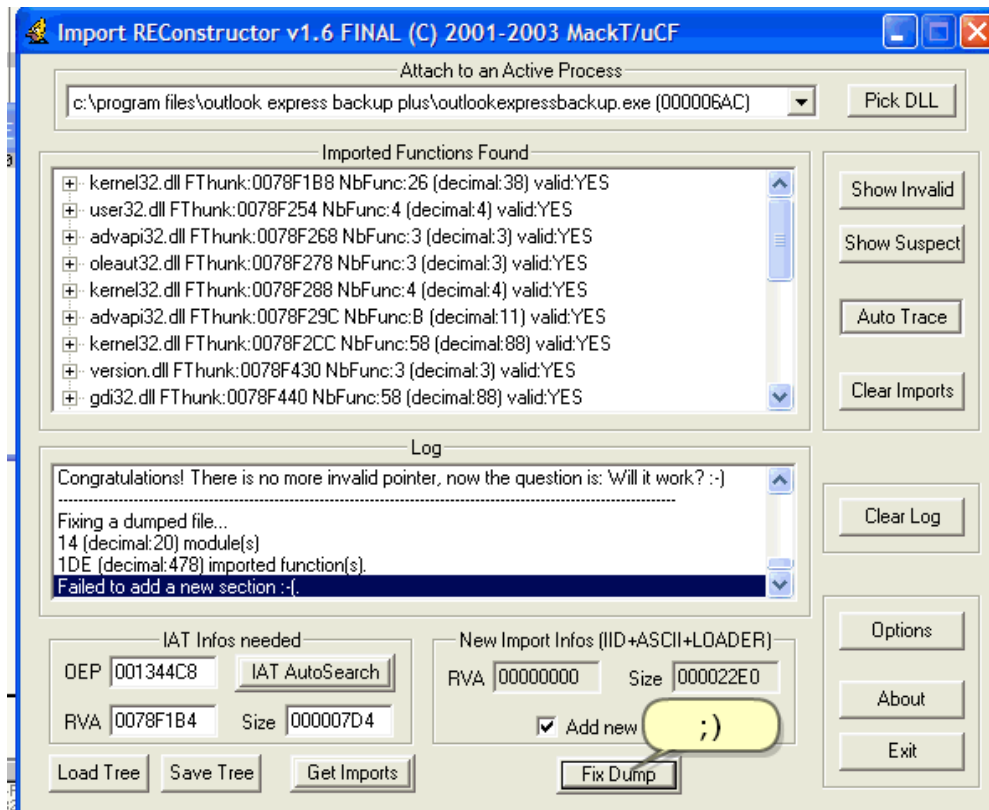
올리 디버거를 새로 하나 실행시켜 원래 바이너리(패킹된 바이너리)를 열어야 한다.

00320000	00006000				Map 0004:
00330000	00006000				Map 0004:
003F0000	00002000				Map 0004:
00400000	00001000	OutlookE		PE header	Imag 0100:
00401000	00134000	OutlookE	CODE		Imag 0100:
00535000	00005000	OutlookE	DATA		Imag 0100:
0053A000	00655000	OutlookE	BSS		Imag 0100:
00B8F000	00003000	OutlookE	.idata		Imag 0100:
00B92000	00001000	OutlookE	.tls		Imag 0100:
00B93000	00001000	OutlookE	.rdata		Imag 0100:
00B94000	00016000	OutlookE	.reloc		Imag 0100:
00B9A000	00030000	OutlookE	.text	code	Imag 0100:
00BDA000	00010000	OutlookE	.adata	code	Imag 0100:
00BEA000	00010000	OutlookE	.data	data,imports	Imag 0100:
00BFA000	00010000	OutlookE	.reloc1	relocations	Imag 0100:
00C0A000	000D0000	OutlookE	.pdata		Imag 0100:
00CDA000	000D9000	OutlookE	.rsrc	resources	Imag 0100:
00DC0000	00103000				Map 0004:

원래 메모리를 보면 PE Header가 있는 걸 볼 수 있다. 해당 부분을 전부 복사해서 현재 바이너리에 붙여넣기를 해준다.

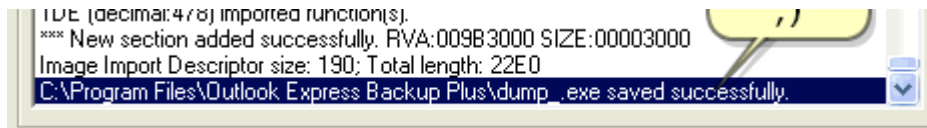


수정했기 때문에 다시 덤프를 떠줘야한다.

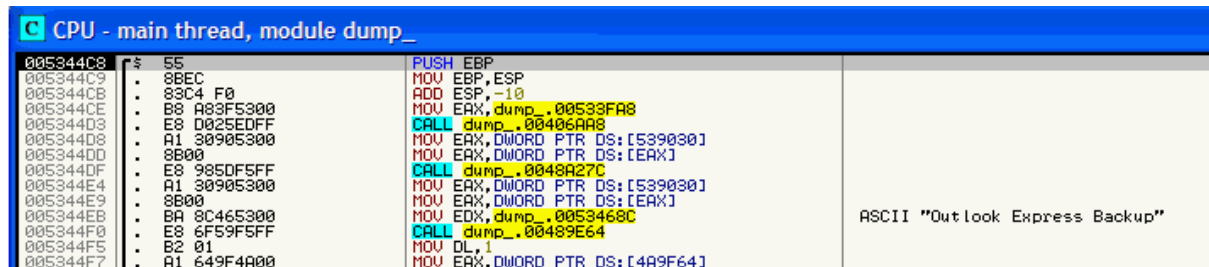


Import REC 틀에 다시 들어가서 Fix Dump를 해준다.

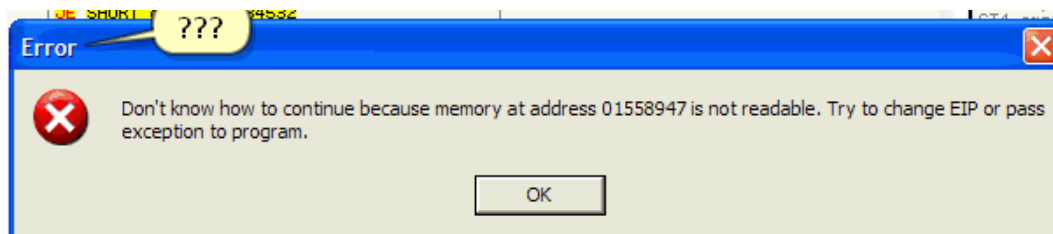
* 이걸 다시 실행해서 수정해 줄 필요 없다. 왜냐하면 Import REC 틀은 실행 중은 부분을 캡처해서 수정하는 틀이기 때문이다. -> 현재 올리디버그로 수정한 부분에 있기 때문이다.



성공한 걸 볼 수 있다.



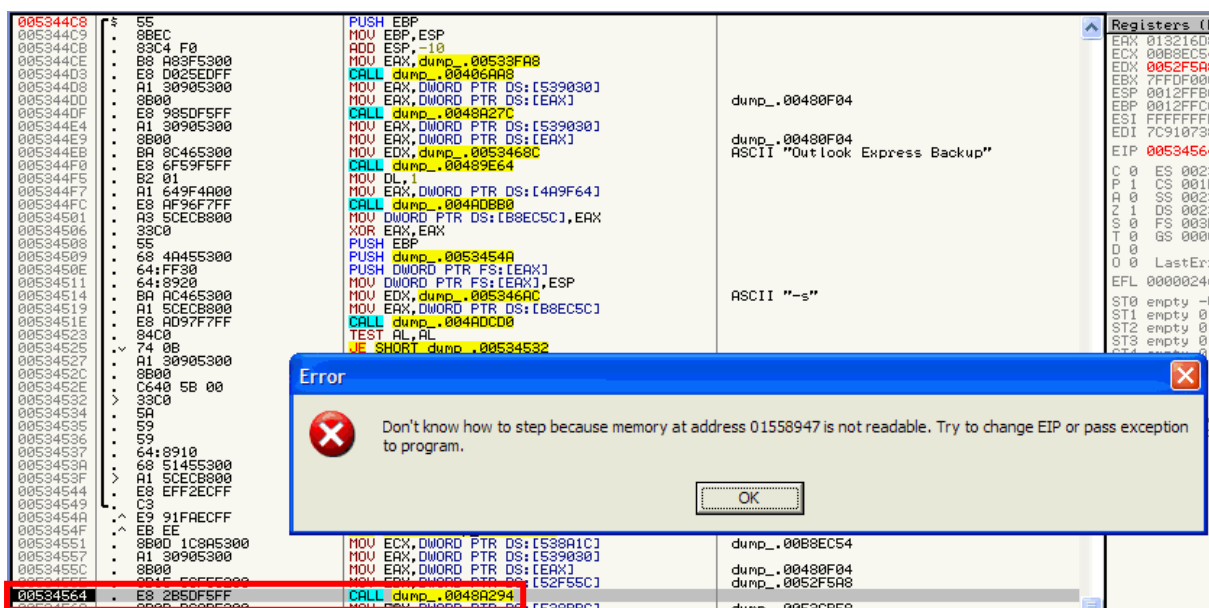
언패킹된 걸 올디버거를 열어준 후 'F9'를 눌러서 다시 실행해본다.



하지만 해당 경고 창이 나오는 걸 볼 수 있다.

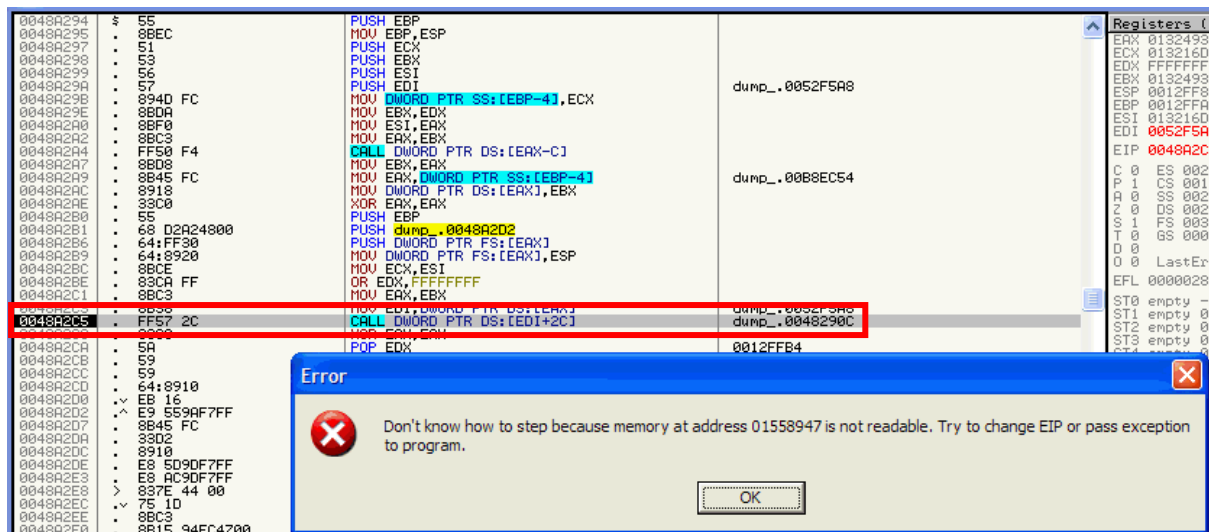
왜냐하면 해당 실행파일은 아르마딜로 패커에 의존을 하는데 해당 코드가 없어져가지고 에러가 발생한다.

'F8'을 눌러서 트레이싱을 해준다.

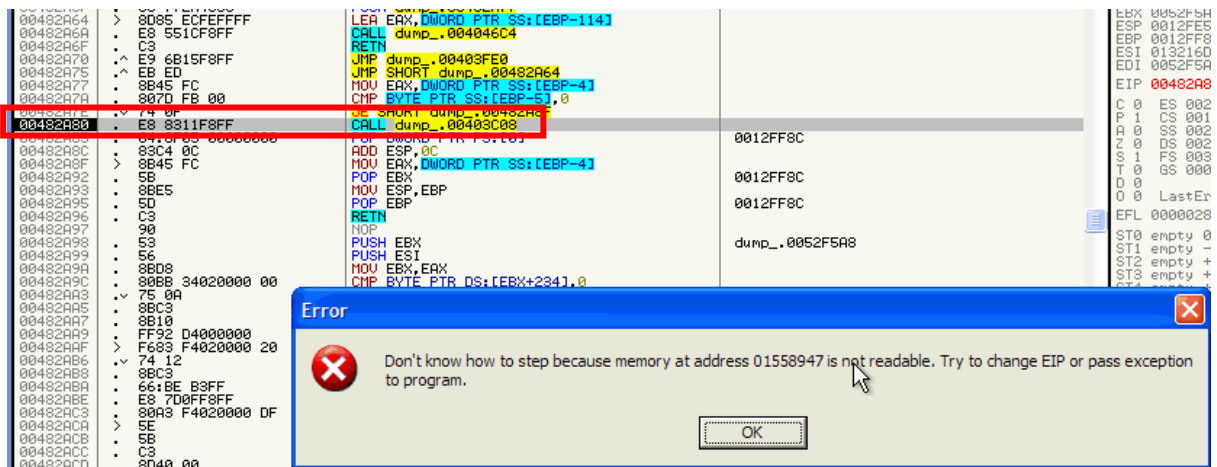


EP로 돌아가서 코드를 하나씩 내려가보면 0x00534564에서 다시 에러가 발생하는 걸 볼

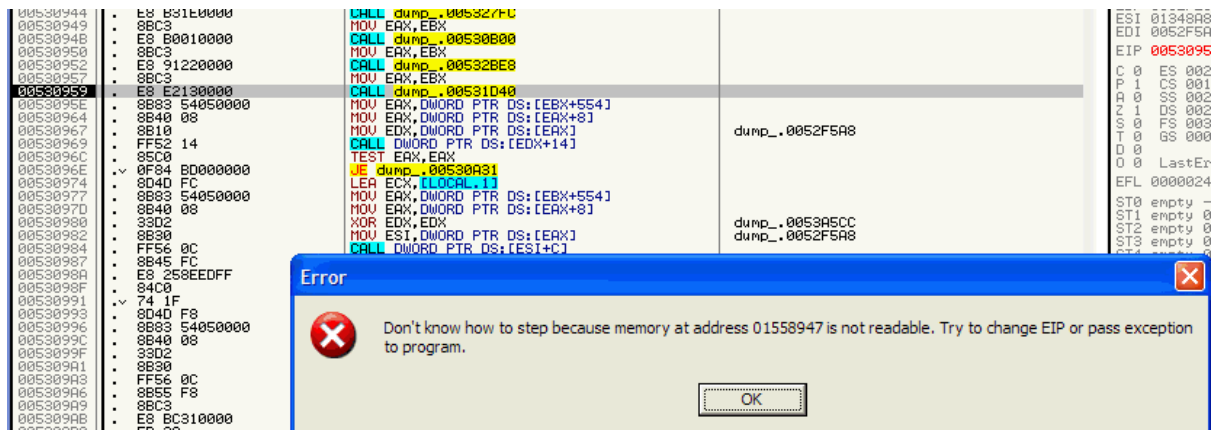
수 있다. 해당 부분에 BP를 걸고 다시 시작한 후 BP 부분에서 'F7'을 이용해서 해당 함수로 들어가준다.



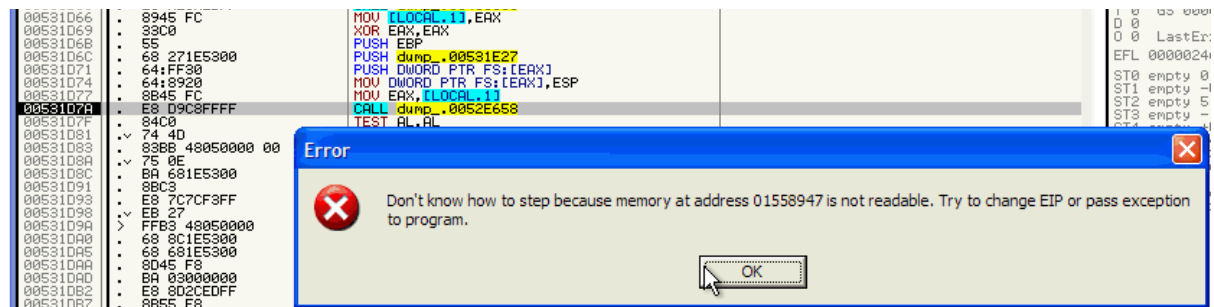
다시 시작한 후 BP 부분으로 돌아가서 코드를 하나씩 내려가보면 0x0048A2C5에서 다시 에러가 발생하는 걸 볼 수 있다. 해당 부분에 BP를 걸고 다시 시작한 후 BP 부분에서 'F7'을 이용해서 해당 함수로 들어가준다.



다시 시작한 후 BP 부분으로 돌아가서 코드를 하나씩 내려가보면 0x0048A280에서 다시 에러가 발생하는 걸 볼 수 있다. 해당 부분에 BP를 걸고 다시 시작한 후 BP 부분에서 'F7'을 이용해서 해당 함수로 들어가준다.

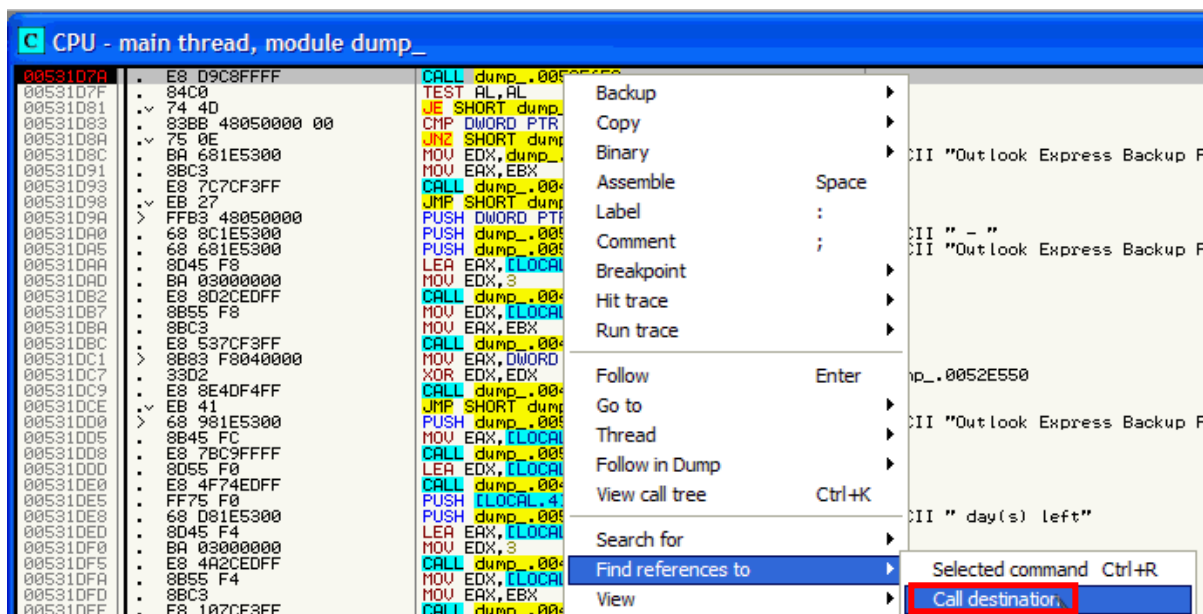


다시 시작한 후 BP 부분으로 돌아가서 코드를 하나씩 내려가보면 0x00530959에서 다시 에러가 발생하는 걸 볼 수 있다. 해당 부분에 BP를 걸고 다시 시작한 후 BP 부분에서 'F7'을 이용해서 해당 함수로 들어가준다.



다시 시작한 후 BP 부분으로 돌아가서 코드를 하나씩 내려가보면 0x00531D7A에서 다시 에러가 발생하는 걸 볼 수 있다. 해당 부분에 BP를 걸고 다시 시작한 후 BP 부분에서 'F7'을 이용해서 해당 함수로 들어가준다.

"015B947"로 분기 되자마자 에러창이 나타나는데, 디버거에 아무런 코드도 보여지지 않는다. 존재하지 않는 주소로 분기하기 때문에 에러가 발생하는 것이다. 따라서 해당 분기가 실행되는 함수를 호출하기 이전 코드 루틴(0x00531D7A)에서 분석을 진행해야 한다.



바로 위 사진의 주소에서 무언가 패치를 해야 한다. 일단 또 다른 주소에서 에러를 발생시키는 "0x00531D7A" 주소를 호출할 수도 있으니 확인해야 한다.

References in dump_:CODE to 0052E658		
Address	Disassembly	Comment
0052F3F8	CALL dump_.0052E658	
0052F490	CALL dump_.0052E658	
00531D7A	CALL dump_.0052E658	(Initial CPU selection)
005322EC	CALL dump_.0052E658	

이렇게 총 4개가 "0x00531D7A" 주소를 호출한다. 4개 전부다 패치를 해줘야한다.

References in dump_:CODE to 0052E658		
Address	Disassembly	Comment
00531D7A	CALL dump_.0052E658	
00531D7F	TEST AL,AL	
00531D81	JZ SHORT dump_.00531D00	
00531D83	CMP DWORD PTR DS:[EBX+5481],0	
00531D8A	JNZ SHORT dump_.00531D9A	
00531D8C	MOV EDX,dump_.00531E68	
00531D91	MOV EAX,EBX	
00531D93	CALL dump_.00469A14	
00531D98	JMP SHORT dump_.00531DC1	
00531D9A	PUSH DWORD PTR DS:[EBX+5481]	
00531DA0	PUSH dump_.00531E8C	
00531DA5	PUSH dump_.00531E68	
00531DAA	LEA EAX,[LOCAL.2]	
00531DA0	MOV EDX,3	
00531DB2	CALL dump_.00404A44	
00531DB7	MOV EDX,[LOCAL.2]	
00531DBA	MOV EAX,EBX	
00531DBC	CALL dump_.00469A14	
00531DC1	MOV EAX,DWORD PTR DS:[EBX+4F81]	
00531DC7	XOR EDX,EDX	
00531DC9	CALL dump_.00476B5C	
00531DCE	JMP SHORT dump_.00531E11	
00531DD0	PUSH dump_.00531E98	
00531DD5	MOV EAX,[LOCAL.1]	
00531DD8	CALL dump_.0052E758	
00531DD0	LEA EDX,[LOCAL.4]	
00531DE0	CALL dump_.00409234	
00531DE5	PUSH [LOCAL.4]	
00531DE8	PUSH dump_.00531E08	
00531DED	LEA EAX,[LOCAL.3]	
00531DF0	MOV EDX,3	

첫 번째 박스의 문자열을 보면 성공했을 때 나오는 문자열인 걸 볼 수 있고 두 번째 박스의 문자열을 보면 실패했을 때 나오는 문자열인 걸 볼 수 있다.

0053107F	B8 00000000	MOV EAX, 0	
00531081	74 4D	JE SHORT dump_.005310D0	
00531083	8B 00	MOV EBX, [EAX]	
0053108C	BA 681E5300	MOV EDI, dump_.00531E68	ASCII "Outlook Express Backup Plus"
00531091	8BC3	MOV EAX, EBX	
00531093	E8 7C7CF3FF	CALL dump_.00469A14	
00531098	EB 27	JMP SHORT dump_.005310C1	
0053109A	FFB3 48050000	PUSH DWORD PTR DS:[EBX+548]	
005310A0	68 8C1E5300	PUSH dump_.00531E8C	ASCII " - "
005310A5	68 681E5300	PUSH dump_.00531E68	ASCII "Outlook Express Backup Plus"
005310AA	8D45 F8	LEA EAX, [LOCAL.2]	
005310AD	BA 03000000	MOV EDI, 3	
005310B2	E8 8B 70F4FF	CALL dump_.00404A44	
005310B7	8B 00	MOV EBX, [LOCAL.2]	
005310BA	8B 00	MOV EBX, [LOCAL.2]	
005310BC	E8 8B 70F4FF	CALL dump_.00469A14	
005310C1	FFB3 48050000	PUSH DWORD PTR DS:[EBX+4F8]	
005310C7	33D0	XOR EDI, EDI	dump_.0052E550
005310C9	E8 8B 70F4FF	CALL dump_.00476B5C	
005310CE	74 41	JMP SHORT dump_.00531E11	
005310D0	68 981E5300	PUSH dump_.00531E98	ASCII "Outlook Express Backup Plus -"
005310D5	8B45 FC	MOV EAX, [LOCAL.1]	
005310D8	E8 7BC9FFFF	CALL dump_.0052E758	
005310DD	8D55 F0	LEA EDI, [LOCAL.4]	
005310E0	E8 4F74EDFF	CALL dump_.00409234	
005310E5	FF75 F0	JMP [LOCAL.4]	
005310E8	68 D81E5300	PUSH dump_.00531ED8	ASCII " day(s) left"
005310ED	8B45 FC	MOV EAX, [LOCAL.1]	

If AL == 1
--> unregistered

The pro
"Unre

해당 분기문에서 판별 되는 걸 볼 수 있다. TEST 명령어에서 판별이 되니까 해당 부분을 바꾸는 걸로 선택하겠다.

0053107A	B8 01000000	MOV EAX, 1
0053107F	84C0	TEST AL, AL
00531081	74 4D	JE SHORT dump_.005310D0

TEST AL, AL -> MOV EAX, 1로 바꿔준다. 아까 총 4개를 패치해줘야 했는데 다 들어가서 바꿔준다.

0052F490	B8 01000000	MOV EAX, 1
0052F495	84C0	TEST AL, AL
0052F497	74 1B	JE SHORT dump_.0052F4B4
0052F499	8D55 F8	LEA EDI, [LOCAL.2]

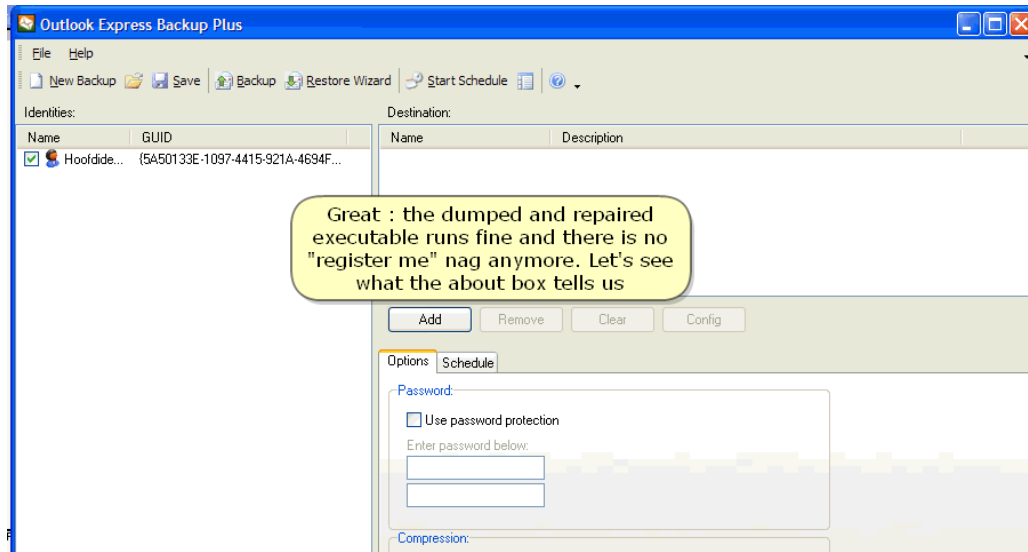
0052F3F8	B8 01000000	MOV EAX, 1
0052F3FD	84C0	TEST AL, AL
0052F3FF	75 16	JNZ SHORT dump_.0052F417

005322EC	B8 01000000	MOV EAX, 1
005322F1	84C0	TEST AL, AL
005322F3	75 49	JNZ SHORT dump_.0053233E
005322F5	A1 D8875300	MOV EAX, DWORD PTR DS:[5387D8]

이렇게 다 TEST AL, AL -> MOV EAX, 1로 바꿔 준 후 새로 저장해준다.

005344C8	55	PUSH EBP	
005344C9	8BEC	MOV EBP, ESP	
005344CB	83C4 F0	ADD ESP, -10	
005344CE	B8 A83F5300	MOV EAX, Test.00533FA8	
005344D3	E8 D025EDFF	CALL Test.00406AA8	
005344D8	A1 30905300	MOV EAX, DWORD PTR DS:[539030]	
005344DD	8B00	MOV EAX, DWORD PTR DS:[EAX]	
005344DF	E8 985DF5FF	CALL Test.0048A27C	
005344E4	A1 30905300	MOV EAX, DWORD PTR DS:[539030]	
005344E9	8B00	MOV EAX, DWORD PTR DS:[EAX]	
005344EB	BA 8C465300	MOV EDI, Test.0053468C	ASCII "Outlook Express Backup"
005344F0	E8 6F59F5FF	CALL Test.00489E64	
005344F5	B2 01	MOV DL, 1	
005344F7	A1 649F4A00	MOV EAX, DWORD PTR DS:[4A9F64]	
005344FC	E8 AF96F7FF	CALL Test.004A0B80	
00534501	A3 5CECB800	MOV DWORD PTR DS:[B8EC5C], EAX	
00534506	33C0	XOR EAX, EAX	
00534508	EB	RET	

새로 저장해 준 파일을 올리디버그로 다시 열어서 실행해준다.



이렇게 제대로 실행되는 걸 볼 수 있다.